Grok:

Resource-Constrained Approach for Track B: BDH-Driven Narrative Reasoning

Team Division and Timeline (6 Days)

- **Day 1-2:** Setup env, clone repos (educational fork: https://github.com/krychu/bdh), download/prep small datasets (TinyStories subsample via HF datasets lib).
- **Day 3-4:** Adapt BDH code for text, pretrain on toys, add classification head.
- **Day 5:** Test on novel chunks, generate visuals.
- **Day 6:** Report, ZIP submission.

Environment Setup

- Python 3.12, PyTorch with CUDA 12.1 (local RTX 3050 for iterations, Kaggle for training).
- Install: torch, transformers, datasets, pathway (for data ingestion).
- Config: n=256-512 neurons, L=4-6 layers, d=128 to fit 6GB VRAM.

Data Preparation

- Novels: Chunk .txt files (512 tokens) via Pathway.
- Pretraining: Subsample TinyStories (10k examples), create synthetic consistent/inconsistent pairs.

Core Approach: BDH-Inspired Sparse Graph for Constraint Tracking

1. **Adapt Educational Fork:**
    - Modify bdh.py/boardpath.py: Replace pathfinding with text inputs (GPT-2 tokenizer).
    - Use Hebbian updates for causal links, sparse state ρ for persistence across chunks.
2. **Pretraining (Kaggle, 2-4 hours):**
    - Objective: Predict next segment; 1-2 epochs, batch=1-4.
    - Merge models for continual learning.
3. **Inference & Classification:**
    - Feed backstory + chunks; embed with activations.
    - MLP head on sparse outputs for 0/1 label.
    - Optional rationale: Utils/ for GIFs showing conflicts.
4. **Long-Context:** Sequential processing with ρ carry-over, Pathway retrieval for relevant chunks.

Novelty & Evaluation Fit

- BDH advantages: Sparse updates for causal tracking vs. Transformer forgetting.

- Report: Compare to GPT-2 baseline, discuss limits (small scale, compute).

Submission

- Code: Runnable script for CSV output.
- Report: 5-8 pages on adaptations, handling, limits.
- Results.csv: Story ID, Prediction, Rationale.

Claude

Track B Approach: BDH-Inspired Sparse State Tracker

Core Strategy (Feasible in 6 days with your resources)

Main Approach: Simplified BDH with Sparse Sequential Processing

Don't try to pretrain BDH from scratch - instead, implement BDH's core principles:

1. **Sequential Chunk Processing with Persistent State**
    - Process novel in 500-1000 word chunks sequentially
    - Maintain evolving hidden state (like RNN but with BDH mechanisms)
    - Each chunk updates the narrative constraint state
2. **Sparse Top-K Updates (BDH's Key Innovation)**
    - At each chunk, compute update gates for all state dimensions
    - Only update the top-k (e.g., k=32) most relevant dimensions
    - This is BDH's breakthrough: selective, sparse updates vs dense transformers
    - Track and report sparsity metrics
3. **Constraint Accumulation Layer**
    - Hidden state accumulates narrative constraints over time
    - Later chunks can reference early constraints through persistent state
    - This enables global consistency checking
4. **Binary Classification Head**
    - Compare final narrative state with encoded backstory
    - Classify: consistent (1) or inconsistent (0)

System Architecture

BDH-Inspired Cell Design

python

```
# Core mechanism (pseudocode)
def bdh_cell(chunk_embedding, previous_state):
    # Compute update importance for all dimensions
    update_gates = sigmoid(W_update @ [chunk, state])

    # BDH KEY: Select only top-k most important updates (sparse)
    top_k_indices = topk(update_gates, k=32)
    sparse_mask = create_sparse_mask(top_k_indices)

    # Apply sparse update
    new_candidate = tanh(W_candidate @ [chunk, state])
    new_state = state * (1 - sparse_mask) + new_candidate * sparse_mask


    return new_state
```

Why This Satisfies Track B Requirements

✅ **"Using BDH principles"**: Sparse top-k updates, persistent state
✅ **"Incremental belief formation"**: State evolves sequentially
✅ **"Clarity vs transformers"**: Sparse (12%) vs dense (100%)

✅ **Feasible on 6GB GPU**: Small model, sequential processing
✅ **Demonstrates understanding**: Can analyze sparsity patterns

---

Implementation Components

1. Data Pipeline (Pathway)

- Use Pathway to ingest novels from Google Drive
- Chunk novels into manageable segments
- Embed with SentenceTransformers (all-MiniLM-L6-v2, 384 dim)

2. BDH-Inspired Model

- Simplified recurrent cell with sparse gating
- Hidden dim: 256 (fits in 6GB GPU)
- Sparsity k: 32 (12.5% active neurons)

3. Training

- Process one novel at a time (memory efficient)
- Batch size: 1
- Mixed precision (FP16) to save memory
- Compare to dense LSTM baseline

4. Evaluation Focus

- **Primary**: Classification accuracy
- **Track B Critical**: Sparsity analysis, comparison to baseline
- Show where BDH helps vs hurts
- Visualize state evolution over narrative

Track B Report Must Include

1. **BDH Principles Section**: Explain sparse top-k gating clearly
2. **Comparison Table**: BDH vs LSTM baseline (accuracy, sparsity, memory)
3. **Sparsity Analysis**: Metrics and visualizations
4. **Ablation Study**: Different k values (16, 32, 64)
5. **Honest Limitations**: What you couldn't implement and why

---

6-Day Execution

- **Days 1-2**: Get minimal BDH cell + data pipeline working end-to-end
- **Days 3-4**: Train BDH model + baseline, optimize hyperparameters
- **Days 5-6**: Generate predictions, write report with analysis

---

Key Success Factor

Track B values **demonstrating understanding of BDH principles** over raw accuracy. Focus on:

- Clear explanation of sparse updates
- Quantitative comparison to dense baselines
- Thoughtful analysis of where/why sparsity helps

GPT:

Project Strategy for a BDH-Inspired Narrative Consistency Classifier

Context: Track B requires determining if a proposed backstory is *consistent* with a given 100k+ word novel. With only 6 days, a 4-person team, no pretrained BDH models, and limited GPUs (Kaggle free tier + RTX 3050 6GB), we must leverage lightweight, memory-efficient methods. Our plan is to **emulate BDH's principles** (sparse attention, persistent memory, incremental reasoning) using practical tools and small models, rather than training BDH from scratch.

BDH Principles & Inspirations

Baby Dragon Hatchling (BDH) is a **biologically-inspired LLM** from Pathway. Key BDH features include: **sparse positive activations** (only a few neurons active at once) and **Hebbian memory** (synapses update during inference)huggingface.colinkedin.com. BDH uses linear-time attention over a high-dimensional "neuron" spacelinkedin.comcolinmcnamara.com and has *no fixed context length* (its "working memory" emerges from dynamic connectivity)colinmcnamara.comhuggingface.co. In practice, BDH rivals GPT-2 on language tasks while offering interpretability and continuous memory

updateshuggingface.cohuggingface.co. We **won't train full BDH**, but we borrow its ideas: use **sparse, local attention** and **persistent memory/"belief" storage** to process very long documents incrementally.

Resource-Efficient Architectures

- **Sparse-Attention Transformers:** Use models like **Longformer** or **BigBird** to handle long context efficiently. Longformer replaces full self-attention with a combination of *local (sliding-window) attention* and a few global tokens, achieving **linear complexity in sequence length**huggingface.co. BigBird similarly mixes local, random, and global attention to process sequences up to ~4096 tokenshuggingface.co. These architectures capture enough long-range context while keeping GPU memory low, aligning with BDH's "sparse" computation.

- **State-Space Models:** Consider Structured State Space (S4) models, which have shown excellent performance on very long sequences (10k+ tokens) with linear-time updatesarxiv.orgarxiv.org. S4 can process arbitrarily long inputs by evolving an internal state, akin to BDH's continuous memory. A lightweight SSM (or similar linear-attention variant) could replace a transformer for summarization or feature extraction on the novel text.

- **Recurrent/Memory-augmented Models:** Use techniques like **Transformer-XL** (segment recurrence) or **Compressive Transformer** to carry hidden state across chunks. These aren't BDH per se but enable *persistent state*. We also draw on research like *"BeliefBank"* which adds an explicit memory of facts to a language modelar5iv.org. In such a system, the model stores answers (beliefs) and can re-query itself to ensure global consistency. We can analogously maintain a "belief" memory of story facts or backstory plausibility scores, updating it as we parse the novel. In practice, we might store sentence embeddings or a small knowledge base that gets updated incrementally (simulating Hebbian updates).

- **Model Pruning/Fine-Tuning:** Given limited GPU memory, we will use **parameter-efficient fine-tuning**. Techniques like **LoRA (Low-Rank Adaptation)** let us adapt large pretrained models by tuning a few low-rank matriceshuggingface.co. For example, using HuggingFace's PEFT library, we can fine-tune a medium-sized model (e.g. DistilBERT or a small T5) by only training a small adapter, saving both memory and timehuggingface.co. This enables training on a single GPU. (In one example, a 3B-parameter FLAN-T5-XL was fine-tuned on a 16GB GPU using LoRA, which would otherwise require multiple 40GB GPUsheidloff.net.) We will leverage such libraries (HuggingFace PEFT, bitsandbytes 8-bit optimizers, etc.) to squeeze performance from small hardware.

BDH-Style Memory and Reasoning

To incorporate BDH-like **persistent memory** and **incremental belief formation**, we propose:

- **Chunked Reading with Memory Update:** Split the novel into manageable segments (e.g. chapters). For each segment, run the model (with sparse attention) and **update a memory vector or database** of key facts. For instance, we could accumulate a compressed summary or vector (using an RNN or pooling model) that carries context forward. This simulates BDH's synapses strengthening: after reading each chunk, we adjust our internal state to "remember" concepts and relationships.

- **Retrieval-Augmented Comparison:** When evaluating a backstory sentence, retrieve relevant context from the novel's memory. Use semantic search (e.g. sentence embeddings) to find novel passages related to each backstory claim. Then use a model (e.g. a smaller transformer or even a GPT-based scoring heuristic) to judge consistency. This is akin to BDH's local graph attention focusing only on relevant neuron neighborhoods.

- **Constraint Reasoning:** Implement lightweight consistency checks. Inspired by BeliefBank, we can have a simple logical/inference stage: e.g., if the backstory contradicts an established fact in memory (via a small SAT-solver or rule engine), flip or flag that contradictionar5iv.org. We can also feed known facts back into the model as context when scoring later claimsar5iv.org. For example, if memory contains "Character A is male," we use that to re-score any backstory sentence claiming the opposite.

- **Sparse Activation Encouragement:** In fine-tuning, we can encourage sparse/monosemantic features (e.g. via attention masks or ReLU activations). While full BDH interpretability is out of scope, using ReLU and attention dropout can lead to sparser activations reminiscent of BDH's positive-only activationshuggingface.co.

Tools and Code Resources

- **HuggingFace Transformers & PEFT:** We will use the Transformers library for model training and inference. Candidate base models include Longformer (for handling long text) and DistilBERT/GPT2 for smaller footprint. HuggingFace's pipeline and Trainer APIs will speed development. We'll use **PEFT/LoRA** (HuggingFace `peft` library) to fine-tune with minimal parametershuggingface.co. Use `bitsandbytes` for 8-bit Adam optimizers and (optionally) QLoRA (4-bit) to lower memory.

- **BDH Repositories (for inspiration):** The official BDH code (pathwaycom/bdh) is PyTorch and demonstrates a toy training loopgithub.com. Community forks (e.g. adamskrodzki/bdh, mosure/burn_dragon_hatchling, severian42/bdh) add features like

dynamic vocab or Apple Silicon supportgithub.com. We can inspect these for implementation ideas (e.g. stateful attention). However, given time constraints, we may only borrow concepts rather than code. The BDH authors even credit nanoGPT code for their toy examplesgithub.com, so using a lightweight GPT-like script is reasonable.

- **Datasets and Preprocessing:** If track B provides labeled data (novel + backstory + label), we'll convert it to HuggingFace `Dataset` format. We may augment data by shuffling or pairing multiple backstories per novel. Preprocess long novels by splitting into sections; for each section create inputs like "**Context:** [section] **Backstory:** [sentence]" for a classification head. Alternatively, preprocess by summarizing chapters (using a pretrained summarizer) and using summaries as context.

- **Baseline Models:** As a baseline, fine-tune a small sequence-classification model (DistilBERT or RoBERTa) on [excerpt, backstory] pairs for consistency. Even this strong baseline (with limited context) provides a reference score. Then we'll layer in the BDH-inspired enhancements (memory and sparse attention) to exceed it.

Proposed Pipeline & Architecture

1. **Preprocess Novel Text:** Split each novel into ordered chunks (e.g. paragraphs or chapters). Optionally, run a fast summarizer (e.g. DistilBART) to compress chunks if needed. Build a **retrieval index** (e.g. using embeddings from SBERT or FAISS) over these chunks for quick lookup.

2. **Memory Vector Initialization:** Initialize an internal "memory" (a vector or knowledge store). This might be as simple as a running aggregate of chunk embeddings, or a small neural memory (e.g. an RNN state).

3. **Iterative Reading Loop:** For each chunk of the novel:

    - Encode the chunk with a transformer (Longformer/encoder model).

    - Update the memory: e.g. update an RNN hidden state or add chunk embedding to a replay buffer. (In spirit of Hebbian updates, reinforce concepts that appear frequently.)

    - Optionally run a quick inference: check if any backstory claim contradicts this chunk. If so, adjust a consistency score (flipping a belief) as in BeliefBankar5iv.org.

4. **Backstory Evaluation:** To classify a candidate backstory:

- Tokenize the backstory (sentence or paragraph). Break into propositions if needed.

- **Retrieve Context:** For each proposition, retrieve the top-N relevant chunks from the novel (using the index).

- **Augmented Input:** Form input sequences like `[RETRIEVED_CHUNKS] + [BACKSTORY_CLAIM]` and feed into a fine-tuned model. Use a sparse-attention model so this can handle multiple chunks. Alternatively, process each chunk-claim pair separately and combine scores.

- **Memory Fusion:** Inject our memory vector into the model. Practically, this could mean concatenating the memory vector to the hidden state, or appending a "[MEMORY]" token whose embedding is the memory. This simulates BDH's internal state influencing attention.

- **Score Aggregation:** Combine scores across claims (e.g. average likelihood or a learned aggregator) to produce a final "consistent/inconsistent" prediction.

5. **Learning:** Fine-tune the classification model on labeled examples. Use a loss that encourages the model to use retrieved context. We may also add auxiliary losses (e.g. contrastive loss so that similar facts produce similar embeddings) to stabilize memory.

6. **Inference:** At test time, follow the same pipeline without gradient steps. Leverage **gradient checkpointing** and mixed precision to fit within GPU VRAMhuggingface.co.

Throughout this pipeline, we emphasize **sparse interactions** (only a few chunks attended at a time) and **state persistence** (memory carried across chunks). This mirrors BDH's local neuron updates and persistent synapseshuggingface.coar5iv.org.

Training and Evaluation Strategy

- **Data Split & Metrics:** Use cross-validation on provided data (if any). For classification, measure accuracy/F1. Also check consistency: if backstory is inconsistent by more than one contradictory claim, penalize more.

- **Fine-Tuning:** Start with a strong pretrained model (e.g. a BERT or Longformer). Freeze most layers and train a small adapter (LoRA). Enable mixed precision (`fp16=True`) and gradient checkpointing to reduce memoryhuggingface.co. Use small batch sizes with gradient accumulation if neededhuggingface.co.

- **Ablation:** Test baseline (no memory, no retrieval) vs. full pipeline to quantify gains from BDH-inspired features. For example, compare (a) simple classification on first 512 tokens of novel, (b) retrieval-augmented model without memory, (c) full incremental memory model.

- **Validation:** If data is scarce, use data augmentation: e.g. swap backstories among novels to create negative examples. Evaluate on withheld test set by Kaggle.

Long-Context Efficiency Tips

- **Sparse Attention Models:** By design, Longformer/BigBird use linear-memory attentionhuggingface.cohuggingface.co, enabling thousands of tokens. We will use their pretrained variants (e.g. `allenai/longformer-base-4096`).

- **Summarization & Chunking:** Instead of feeding 100k tokens at once, process in chunks. Summarize or encode each chunk separately, then merge. This reduces peak memory.

- **PEFT & Quantization:** Use **LoRA/QLoRA** to only train a few parametershuggingface.co. Use `bitsandbytes` 8-bit or 4-bit optimizers (as HuggingFace supports) to shrink model size. A 7B model can often be 4-bit quantized and fine-tuned on a 6GB card if only a small rank is trainedheidloff.net.

- **Gradient Checkpointing:** Enable checkpointing to save memory at the cost of some speedhuggingface.co. For example, setting `gradient_checkpointing=True` in HuggingFace's Trainer can halve memory usage.

- **Batching and Accumulation:** Use tiny `per_device_train_batch_size` (e.g. 1–2) with `gradient_accumulation_steps` to mimic larger batches without needing the memory at oncehuggingface.co.

- **Mixed Precision:** Use FP16 (`trainer = Trainer(..., fp16=True)`) for faster and leaner traininghuggingface.co. On Ampere GPUs (like Kaggle's P100/V100), TF32/BF16 can also be enabled.

- **Profile Early:** Continuously monitor GPU usage on Kaggle and local. Use `nvidia-smi` or HF's tools to avoid OOM. Adjust model size (e.g. switch from base to distil or reduce layers) if needed.

Actionable Steps

1. **Gather Resources:** Clone BDH repos for referencegithub.comgithub.com. Set up Kaggle and local env (install `transformers`, `peft`, `datasets`, `bitsandbytes`).

2. **Baseline Model:** Fine-tune a small pre-trained classifier (e.g. DistilBERT) on [novel-excerpt, backstory] pairs to get a starting score.

3. **Implement Retrieval:** Compute embeddings for novel chunks (e.g. with Sentence-BERT) and build a search index (e.g. FAISS). Test retrieving relevant passages given a backstory query.

4. **Memory Module:** Prototype a simple memory: e.g. an RNN state that takes chunk embeddings as input. Ensure it improves performance on a small dev set.

5. **Sparse Attention Model:** Switch to Longformer/BigBird for direct chunk processing. Fine-tune it (with LoRA) on the task, feeding multiple chunks if possible.

6. **Pipeline Assembly:** Combine retrieval + memory + long-model. Create an inference loop that reads novel, updates memory, and scores a full backstory.

7. **Optimize Performance:** Apply FP16, checkpointing, and LoRA. Profile GPU memory and speed. Iteratively reduce model size or batch size as needed.

8. **Evaluation & Tuning:** Validate on held-out data. Tune retrieval chunk size, memory update frequency, and ensemble techniques (e.g. voting across multiple chunk scores).

9. **Documentation & Edge Cases:** Document how BDH principles are used (sparse/memory). Prepare for potential questions on why this is competitive (e.g. why BDH ideas help long-story context).

By focusing on **efficient long-context processing** and **incremental memory** (as in BDH), we maximize performance under constraints. With careful engineering (PEFT, sparse attention, retrieval, and memory), we can build a competitive classification system that embodies the spirit of BDH without requiring huge compute.

Sources: We draw on BDH's design and related researchhuggingface.colinkedin.comar5iv.org, as well as known efficient-model techniqueshuggingface.cohuggingface.coarxiv.orghuggingface.co. These guide our architecture and tool choices.

Gemini:

The Core Strategy: "State-Space Surprisal"

Since you cannot fit the entire novel into context using standard attention (it requires too much VRAM), you will use BDH as a **Recurrent Neural Network**.

The Logic:

1. **Read the Novel:** Feed the novel into BDH. As it reads, it updates a fixed-size "Hidden State" (h_t). By the end of the book, this state mathematically represents the "world model" of that specific story.
2. **Test the Backstory:** Feed the backstory into the model *initialized* with that final world state.
3. **Measure "Surprise":** Calculate the perplexity (loss).
    - If the backstory is **Consistent**, the model will find it predictable (Low Loss).
    - If the backstory is **Inconsistent**, the model will be "surprised" by the contradictions (High Loss).

Step-by-Step Implementation

Phase 1: Setup (Day 1)

- **Codebase:** Clone the **Educational Fork** [1] mentioned in the problem statement (adamskrodzki/bdh or similar from the document).
    - *Why?* It includes visualization tools (utils/) and is lighter than the main repo, making it perfect for your RTX 3050.
- **Environment:**
    - **Local (RTX 3050):** Use for debugging and generating the visualization GIFs.
    - **Kaggle (2x T4):** Use for the heavy data processing/inference loops.

Phase 2: The Pipeline (Days 2-4)

1. **Ingestion (Pathway):**
    - Use **Pathway** to stream the text files[2]. Do not load the whole 100MB text into RAM at once.
    - *Requirement:* Track A requires Pathway, but using it in Track B simplifies your data loading significantly.
2. **The "Reader" (Model Inference):**
    - **Configuration:** Set BDH to **Recurrent Mode** (Linear/State-Space mode). This ensures $O(1)$ memory usage per token.
    - **Training vs. Inference:** You likely don't have time/compute to train a model from scratch to high English proficiency.
    - **Hack:** Use a small pre-trained model if available in the repo, or train a *tiny* model (dim=256) strictly on the provided novels to learn character names and basic relationships.
    - **Execution:**
        - Pass the **Novel** token-by-token.
        - **Important:** Keep the state variable persistent across chunks. Do not reset it.
        - Save the final state vector (S_{final}).

3. **The "Judge" (Scoring):**
    - Load S_{final}.
    - Pass the **Backstory** text.
    - Compute **Cross-Entropy Loss** for the backstory tokens.
    - Normalize by length to get a "Consistency Score".

Phase 3: Visualization & Report (Day 5)

- **Visuals:** The problem statement explicitly values "Interpretability findings" and "sparsity measurements"[3].
- **Action:** Use the educational fork's tools to generate a **Heatmap GIF**[4].
    - Show the neuron activations being stable during the novel.
    - Show them "spiking" (red hot) when the model reads the contradictory part of the backstory.
    - *This visual proof is your "Evidence Rationale" without needing complex text retrieval.*

Phase 4: Submission (Day 6)

- **Thresholding:** You will get a score (e.g., 4.5, 3.2, 8.1) for each backstory.
- **Calibration:** Use the provided labeled examples to pick a cutoff number. (e.g., "Any score > 5.0 is Inconsistent").
- **Deliverables:** Code, Report (focusing on the "Surprisal" method), and results.csv[5].

Why This Strategy Wins Track B

1. **Hardware Safe:** Recurrent mode fits 100k context on 6GB VRAM. Standard Transformers (Track A approach) would crash your GPU immediately.
2. **Matches Criteria:** It focuses on "how BDH-style mechanisms affect learning... over long narratives"[6].
3. **Feasible:** It avoids complex "Agentic" workflows or massive RAG pipelines, which are time-consuming to build.

Perplexity:

Overall strategy

- Use BDH as a compact recurrent encoder to turn a long novel + its backstory into fixed-size representations.
- Train a small classifier head on top of these representations to predict consistency (0/1).
- Keep the BDH part small and stateful, not a huge LLM, so it fits on 6 GB VRAM and Kaggle free GPUs.

2. Data and long-context handling

- Tokenize each novel and backstory with a BPE tokenizer compatible with your BDH implementation.
- Chunk each novel into sequences of ~1k–2k tokens.
- Process chunks sequentially:
    - Maintain BDH's internal state (neurons + synapses) across chunks to model evolving beliefs over the narrative.
    - After the last chunk, treat the final BDH state as the novel representation.
- Encode the backstory text with the same BDH (separate pass) to get a backstory representation.

---

3. BDH-style encoder design

- Start from the official BDH / BDH-GPU implementation.
- Build a small model:
    - Neuron count and layers chosen so a forward pass on a 1–2k-token chunk fits in 6 GB with fp16.
- Preserve BDH-specific ideas:
    - Recurrent state across tokens (no KV cache) and across chunks.
    - Sparse positive activations (e.g., ReLU + threshold or top-k) to mimic BDH's interpretable sparsity.
    - Hebbian-like synaptic updates (outer product with decay) as working memory.
- Option: initialize from small BDH weights if provided, otherwise train a lightweight encoder from scratch on your dataset or a subset of related text.

---

4. Classifier and training objective

- After encoding:
    - Concatenate [novel_state, backstory_state, similarity_features].
    - Similarity features can be cosine similarity, elementwise difference, etc.
- Feed this vector into a small MLP (e.g., 1–2 hidden layers) that outputs a consistency probability.
- Train with:
    - Binary cross-entropy loss on the provided labels.
    - Mixed precision + gradient accumulation to cope with small GPU.

---

5. Baseline & ablations (for report and sanity)

- Build a non-BDH baseline:
    - Use a small sentence embedding model (e.g., bge-small-en) to embed:
        - Sampled or pooled novel text.
        - Backstory text.
    - Train the same MLP classifier on top.
- Compare BDH vs baseline:
    - Accuracy on validation set.
    - Stability when novels are truncated or shuffled.
- Use these results to argue how BDH-style state and Hebbian memory affect performance, as required for Track B.

---

6. 6-day work split (very high level)

- Day 1–2: Set up data pipeline, tokenization, chunking; get small BDH encoder running on one novel.
- Day 3: Implement full stateful pass over novel + backstory, extract representations.
- Day 4: Implement MLP classifier, run first training; adjust model size for 6 GB.
- Day 5: Add sparsity + Hebbian updates, tune hyperparameters; build simple transformer/encoder baseline.
- Day 6: Final training runs (Kaggle), generate test predictions, write report focusing on BDH mechanisms and long-context reasoning.